

IN1101 - Programming Fundamentals

Lab Session 01

Introduction to C Programming & Environment Setup

W.A.G.Nilnuwani

Lecturer,

Department of Information Technology,

Faculty of Information Technology,

University of Moratuwa

Lab Objectives

By the end of this lab, you will be able to:

- Understand the **structure of a C program**
- Write and execute a **basic C program**
- Compile and run C programs using **GCC / IDE**
- Use **printf() with escape sequences**
- Identify **common errors in C programs**

What Is C Programming?

- C is a structured, procedural programming language
- Very close to hardware
- Used to develop:
 - Operating Systems
 - Embedded Systems
 - Compilers
- Learning C improves logical thinking

Structure of a C Program

- C program consists of:
 - Documentation Section
 - Link Section
 - Definition Section
 - Global Declaration Section
 - Main Function
 - User Defined Functions (Sub Programs)

Documentation Section (Comments)

- Used to describe:
 - Program owner
 - Created date
 - Purpose of the program
- Single-line comment
// Created by Nilnuwani
- Multi-line comment
/* Created by Nilnuwani
Date: 02.02.2026
*/

Compiler does **not** execute
comments!

Link Section (Header Files)

- Used to include **library functions**

```
#include <stdio.h>
#include <math.h>
#include <string.h>
```

Header files
always end
with **.h**

Header File	Purpose
stdio.h	printf(), scanf()
math.h	sqrt()
string.h	strlen()

Preprocessor in C

- **#include**, **#define** are handled by Preprocessor
- Preprocessor runs before compilation
- It performs text substitution
- Preprocessor is **not** part of the compiler

Definition Section

- Used to define **symbolic constants**

```
#define PI 3.14159
```

- Value cannot be changed
- Improves code readability
- Used multiple times safely

Global Declaration Section

```
int total; // global variable
```

Variable Type	Defined
Local	Inside a function
Global	Outside all functions

Main Function

- Execution of a C program **starts from main()**

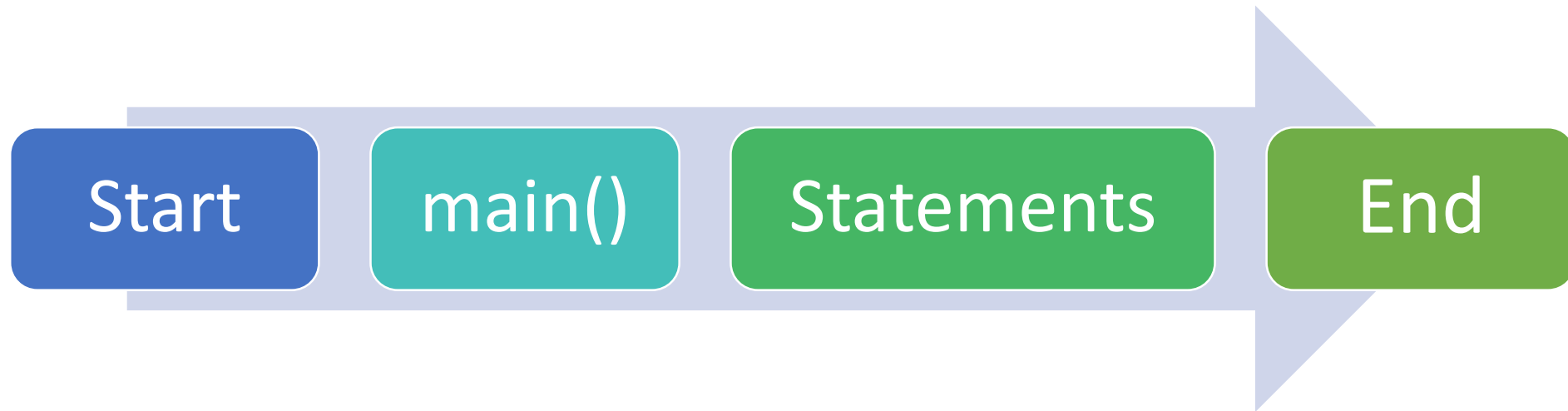
```
#include <stdio.h>

int main() {
    int a = 5, b = 6;
    int sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```



Statements
execute line
by line !

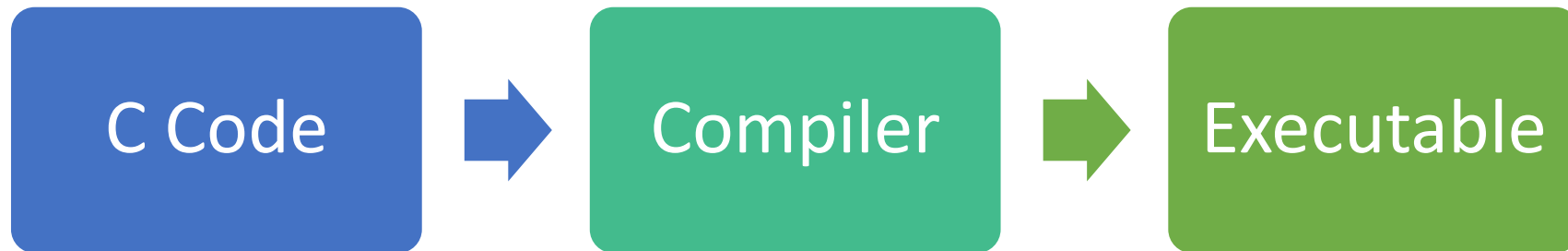
Program Execution Flow



- There can be many functions
- Execution always starts from **main()**

What Is a Compiler?

- Converts C source code → Machine code
- Produces an executable file



What Is GCC?

- GCC – GNU Compiler Collection
- Open-source compiler
- Converts **.c** files into **.exe**

```
gcc hello.c -o hello
```

- **hello.c** → source file
- **hello** → executable file

What Is MinGW?

- MinGW – Minimalist GNU for Windows
- Brings GCC to Windows
- Includes:
 - gcc.exe (C compiler)
 - g++.exe (C++ compiler)

First C Program (Hello World)

```
#include <stdio.h>

int main() {

    printf("Hello, World!\n");
    return 0;

}
```

- ✓ Write
- ✓ Compile
- ✓ Run

Compile & Run Using Command Prompt

- **Step 1: Open a Text Editor**
- Open **Notepad** (or any text editor)
- Type the following C program:

```
#include <stdio.h>
int main() {

    printf("Hello, World!\n");
    return 0;
}
```


Compile & Run Using Command Prompt

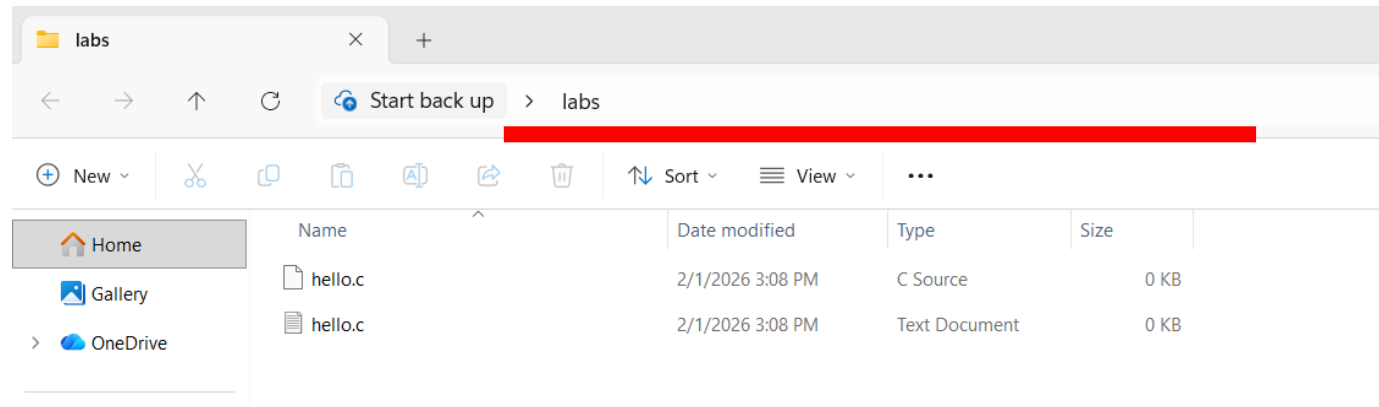
- **Step 2: Save the File as hello.c**
- Click **File → Save As**
- Choose a folder (e.g. Desktop or your lab folder)
- In **File name**, type: **hello.c**
- Click **Save**

Important!

File extension must be **.c**
Do **not** save as **hello.c.txt**

Compile & Run Using Command Prompt

- **Step 3: Open Command Prompt**
- Click the address bar in your folder



- Type: **cmd**
- Press **Enter**
- A black Command Prompt window will open

Compile & Run Using Command Prompt

Step 4: Compile the C Program

- Type the following command and press **Enter**:

```
gcc hello.c -o hello
```

- gcc → C compiler
- hello.c → source file
- -o hello → output executable file

If there are **no errors**,
compilation is successful!

If errors appear, check the
code carefully

Compile & Run Using Command Prompt

- **Step 6: Run the Program**

- Type:

```
hello
```

and press **Enter**

- **Step 7: View the Output**

- Output:

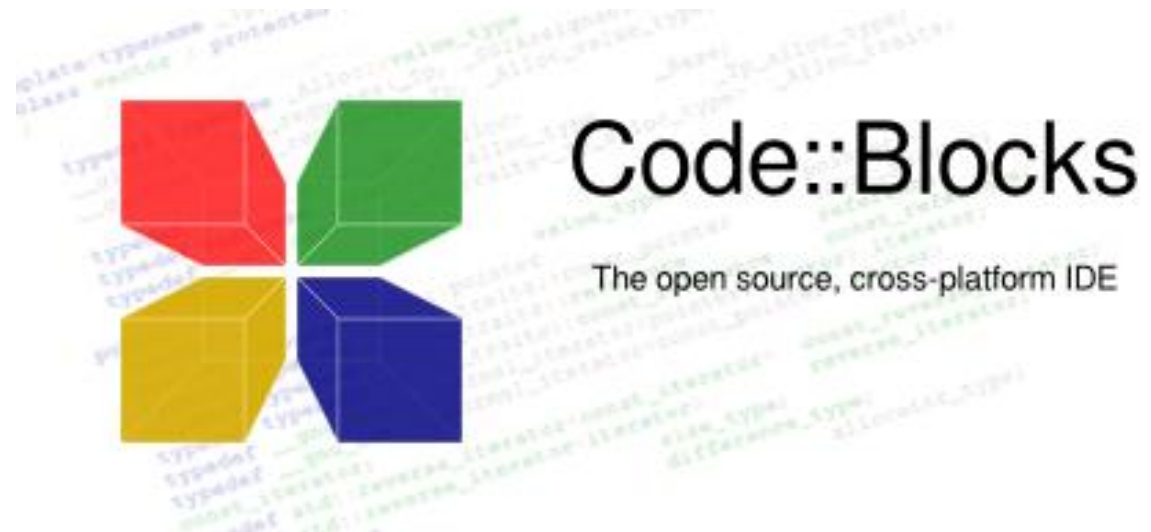
```
Hello, World!
```



You have
successfully
**compiled and
executed your
first C program!!!**

What Is an IDE?

- **IDE – Integrated Development Environment**
- An IDE includes:
 - Code Editor
 - Compiler
 - Debugger
 - Build & Run tools
- Example: **Code::Blocks**



Write and Run the First C Program Using Code::Blocks

Step 1: Open Code::Blocks

- Click Start Menu
- Search for Code::Blocks
- Open the application
- Wait until Code::Blocks loads completely

Step 2: Create a New File

- Click File → New → Empty File
- A blank editor window will open
- This file will contain your C program

Write and Run the First C Program Using Code::Blocks

- **Step 3: Save the File as hello.c**
- Click **File** → **Save**
- Choose a suitable folder (e.g. Desktop / Lab Folder)
- Enter file name: **hello.c**
- Click **Save**
- Ensure the file extension is **.c**

Write and Run the First C Program Using Code::Blocks

Step 4: Write the C Program

- Type the hello world program code in the editor

Step 5: Build the Program

- Click **Build** → **Build**
- Press **Ctrl + F9**
- This step checks for syntax errors

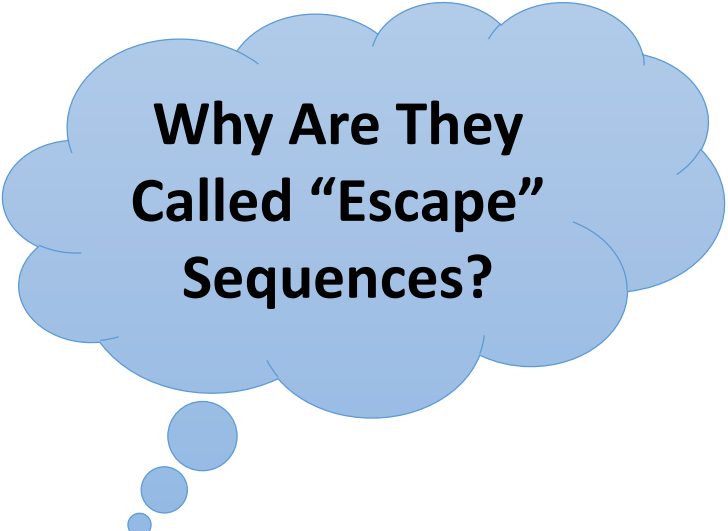
Step 6: Run the Program

- Click **Build** → **Run** OR
- Press **Ctrl + F10**
- A console window will open
- View the output

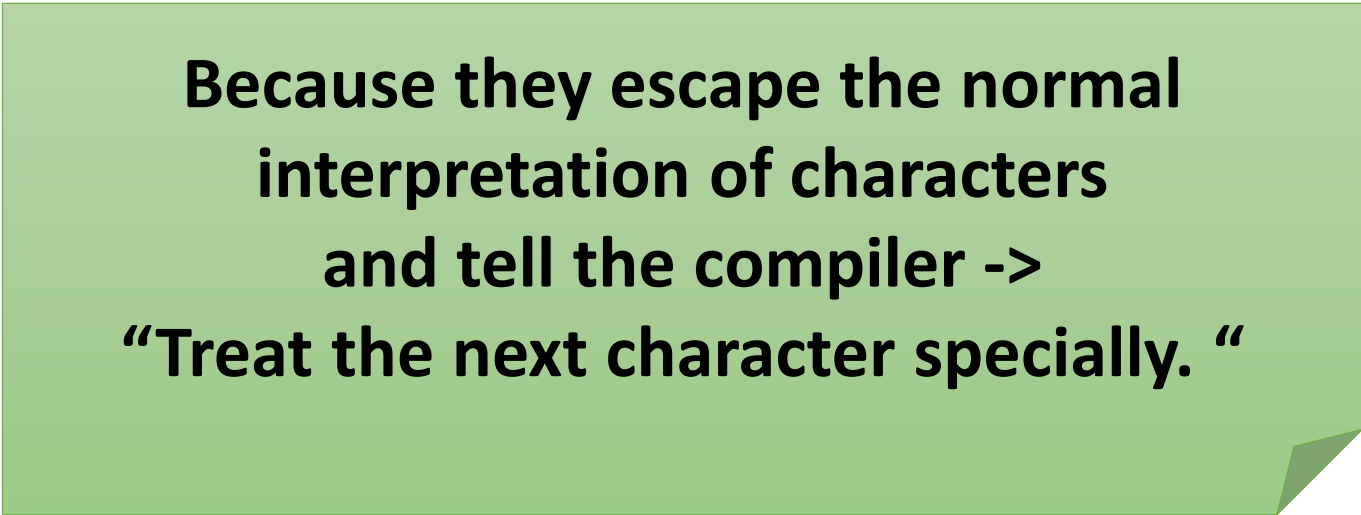
Hello, World!

What Is an Escape Sequence?

- An escape sequence is a special character used inside a string literal that:
- Starts with a backslash (\)
- Represents characters that cannot be typed directly or have special meaning



**Why Are They
Called “Escape”
Sequences?**



**Because they escape the normal
interpretation of characters
and tell the compiler ->
“Treat the next character specially. “**

Common Escape Sequences

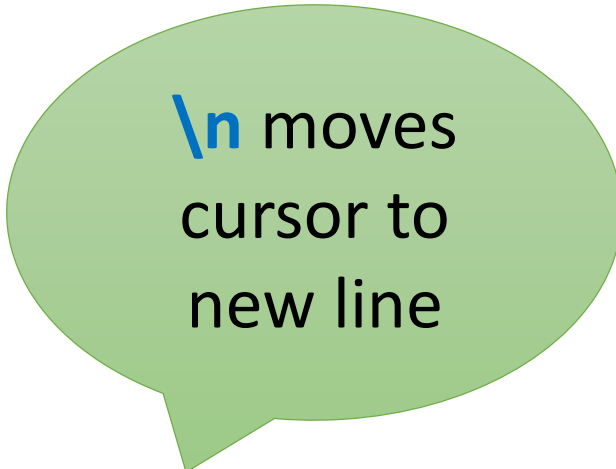
Escape	Meaning
<code>\n</code>	New line
<code>\t</code>	Tab
<code>"</code>	Double quote
<code>'</code>	Single quote
<code>\</code>	Backslash
<code>\b</code>	Backspace
<code>\r</code>	Carriage return
<code>\0</code>	Null character

Escape Sequence - \n — New Line

```
#include <stdio.h>

int main() {
    printf("Hello World\n");
    printf("Welcome to C Programming");

return 0;
}
```



`\n` moves
cursor to
new line

Escape Sequence - \t — Horizontal Tab

```
#include <stdio.h>

int main() {
    printf("Mathematics\t90\n");
    printf("English\t\t80\n");

    return 0;
}
```



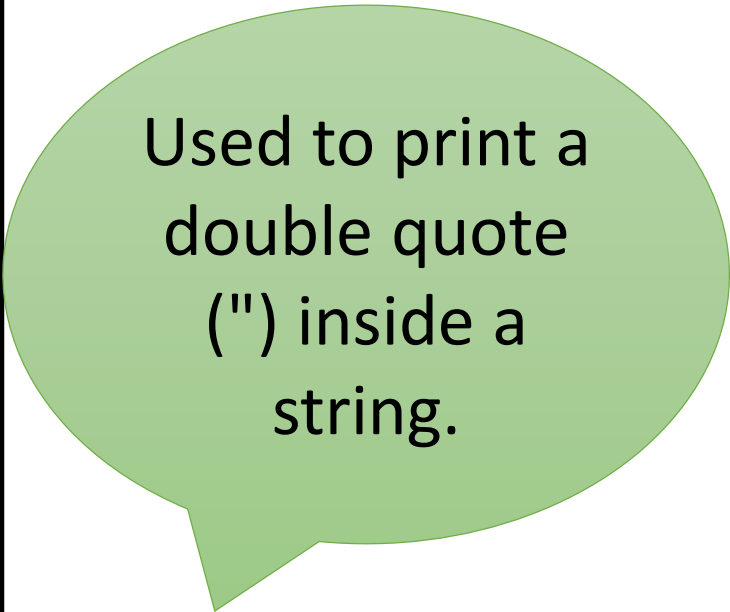
\t adds
horizontal
spacing

Escape Sequence - \" — Double Quote

```
#include <stdio.h>

int main() {
    printf("She said, \"C is powerful!\"");

return 0;
}
```



Used to print a double quote (") inside a string.

Escape Sequence - \' — Single Quote

```
#include <stdio.h>

int main() {
    printf("It is called \'C programming\'");
    return 0;
}
```

Escape Sequence - \\ — Backslash

```
#include <stdio.h>

int main() {

printf("C:\\Users\\Student\\Documents");

return 0;
}
```

- One backslash is escape character
 - Two backslashes print one backslash

Escape Sequence - \b — Backspace

```
#include <stdio.h>

int main() {

printf("AB\bC");

return 0;
}
```

Moves the cursor
**one position
backward**
and removes the
previous character.

Escape Sequence - \r — Carriage Return

```
#include <stdio.h>

int main() {

printf("12345\rABC");

return 0;
}
```

Moves the cursor to the **beginning of the same line** (overwrites existing characters).

Escape Sequence - \0 — Null Character

```
#include <stdio.h>

int main() {
    printf("Hello\0World");

    return 0;
}
```

Represents **end of string.**

Types of Errors in C

- C programs can have:

Syntax
Errors

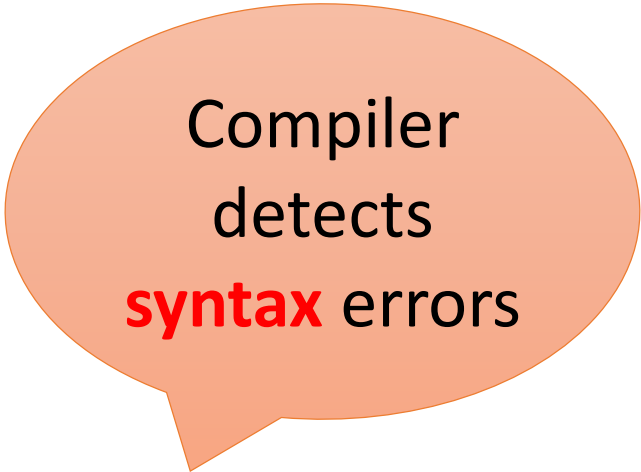
Semantic
Errors

Runtime
Errors

Syntax Errors

- Grammar mistakes
- Program does **not compile**

```
int a = 5 // missing semicolon
```



Compiler
detects
syntax errors

Semantic Errors

- Logical mistakes
- Program compiles but output is wrong

```
sum = a - b; // wrong logic
```



Most
dangerous
type of error

Runtime Errors

- Occur during execution
- Program may crash
- Example: Division by Zero

```
int a = 10, b = 0;  
printf("%d", a / b);
```

Lab Exercise 01

- Write a C program to display

Welcome to University of Moratuwa
Faculty of Information Technology
Department of Information Technology

- Use multiple printf() statements.

Lab Exercise 02

Write a C program to print:

- Full Name
- Index Number
- Degree Program
- Each item should appear on a **new line**.

Lab Exercise 03

Print the following using only ONE printf():

```
C Programming  
is  
fun!
```

Hint: Use escape sequences.

Lab Exercise 04

Write a C program to display:

Subject	Marks
Mathematics	90
English	85
Science	88

Hint: Use `\t` for alignment.

Lab Exercise 05

Write a C program to display:

```
She said, "C programming is powerful!"
```

Hint: Use escape sequences correctly.

Lab Exercise 06

Write a C program to print:

```
C:\Users\Student\Documents\C_Labs
```

Lab Exercise 07

Write a C program that includes:

- Single-line comments - This program demonstrates comments in C
- Multi-line comments - Created by Student, Date: 02.02.2026
- Prints a message :
"This program demonstrates comments in C"

Lab Exercise 08 - I

Without running the program, predict the output and then run, verify.

```
#include <stdio.h>
```

```
int main() {
```

```
printf("ABCDE\r123\rXY");
```

```
    return 0;
```

```
}
```

Lab Exercise 08 - II

Without running the program, predict the output and then run, verify.

```
#include <stdio.h>
```

```
int main() {
```

```
printf("ABC\rDEF\nGHI");
```

```
    return 0;
```

```
}
```

Lab Exercise 08 - III

Without running the program, predict the output and then run, verify.

```
#include <stdio.h>
```

```
int main() {
```

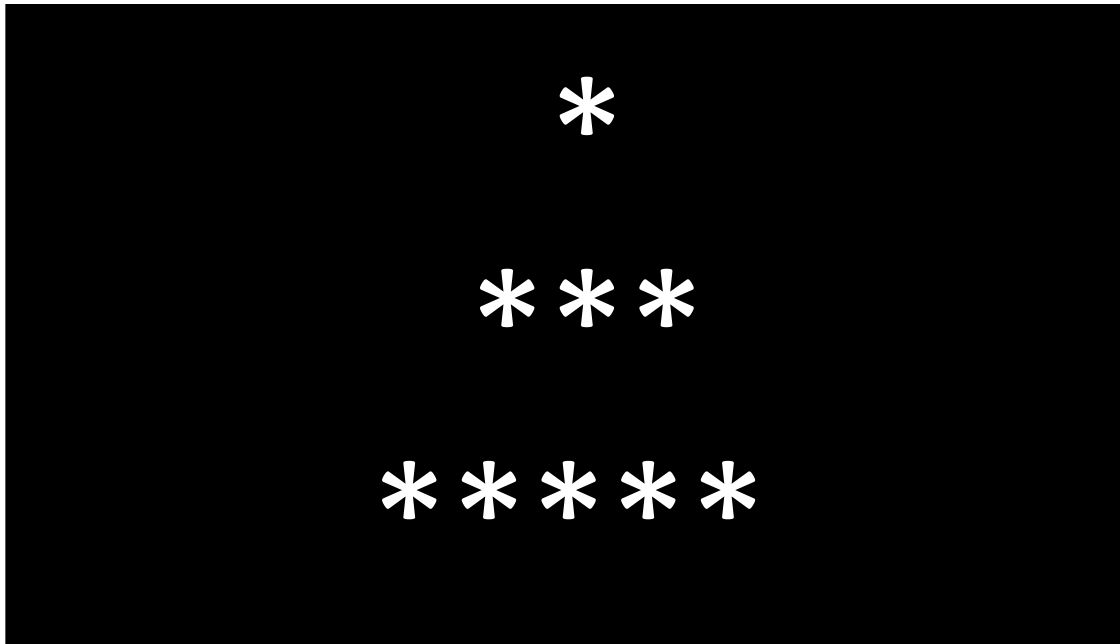
```
printf("Name\tMarks\rJohn\t90");
```

```
    return 0;
```

```
}
```


Lab Exercise 09

Print the following pattern:



Use spaces and * characters.

Lab Exercise 10

- Create a C program that prints a **simple welcome screen** using:
- Tabs
- New lines
- Quotes

```
*****
```

```
* WELCOME! *
```

```
*****
```